

TECHNICAL REPORT

JAS OS

A 32-bit x86 Educational Operating System for CSE 323

Student: Jubayed Ahmed Sahel

Student ID: 2221173642

Course: CSE 323 - Operating Systems, Section 3

Version: JAS OS v1.5.3

Date: 21 August 2026

Repository: github.com/Jubayed-Sahel/JAS-OS

Executive Summary

JAS OS is a bootable, freestanding 32-bit x86 teaching operating system. Its purpose is to turn the major CSE 323 concepts from chapters 1 through 13 into observable kernel behavior instead of presenting them only as slide definitions. The system boots from an ISO in Oracle VirtualBox, enters protected mode, initializes interrupt and input hardware, and launches a graphical desktop with a terminal, file manager, notes editor, task manager, calculator, settings, clock, and one-click OS Lab.

The implementation is intentionally honest about its educational scope. Process states, scheduling, semaphores, heap allocation, paging translation, Banker's safety checks, file operations, disk scheduling, and I/O modes are implemented as kernel modules. Paging, disk, RAID, and DMA behavior are controlled teaching models rather than claims of full production hardware support. Chapter 14 onward is excluded, matching the project scope.

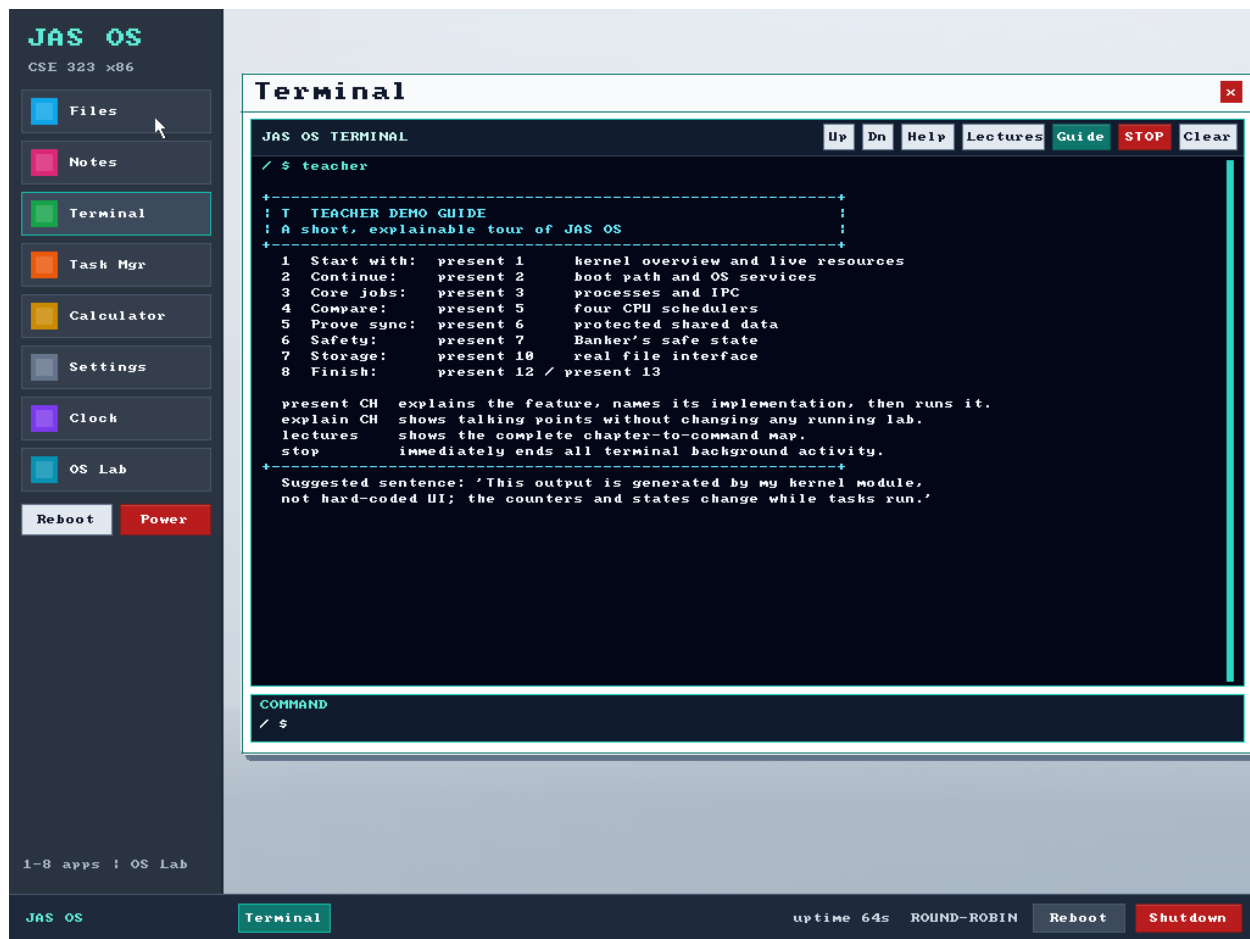


Figure 1. JAS OS teacher-facing terminal running in Oracle VirtualBox.

1. Project Objective and Scope

The original work was an ESP32-S3 mini-kernel. The objective of this project was to preserve its operating-system learning value while moving the experience to an x86 virtual machine with a bootable ISO and a more explainable graphical interface. The result is suitable for a live classroom demonstration: every chapter has a mapped command, live state, and a safe stop path, while the desktop applications make the system usable beyond algorithm output.

Included. Core concepts from chapters 1-13, an x86 boot path, graphical desktop, kernel terminal, presentation mode, and VM-ready ISO.

Excluded. Chapter 14 onward, networking, user-mode protection rings, production device drivers, and claims of production-grade persistence or security.

2. System Architecture

The BIOS loads a custom El Torito boot image. A 16-bit loader reads the kernel, requests a VESA linear framebuffer, enables the A20 line, installs a Global Descriptor Table, and transfers control to 32-bit

protected mode. The freestanding C kernel then initializes the BSS and stack, IDT/PIC/PIT, serial output, PS/2 keyboard and mouse, graphics, scheduler, memory manager, paging model, Banker's state, MiniFS, storage models, event log, and GUI.

Layer	Responsibility
Boot	BIOS -> El Torito image -> 16-bit loader -> VESA/GDT/A20 -> protected mode
Kernel core	IDT, PIC, 1 kHz PIT, serial, PS/2 input, cooperative scheduler
Resource modules	tasks, semaphores, heap, paging, deadlock avoidance, MiniFS, storage/I/O models
Presentation layer	framebuffer GUI, desktop applications, terminal command center, OS Lab

3. Implemented Course Features

Ch.	Topic	JAS OS live implementation
1	OS organization	Live dashboard, resource counters, kernel/system information
2	Services and boot	Boot trace, system-service map, syscall-style command aliases
3	Processes and IPC	Task states, lifecycle control, process table, kernel mailbox
4	Threads	Two schedulable workers sharing a semaphore-protected counter
5	CPU scheduling	Round Robin, Priority, FCFS, SJF, tuning, and CPU timeline
6	Synchronization	Counting/binary semaphores, producer-consumer, readers-writers
7	Deadlocks	Banker's safety test, request/release, and safe sequence
8	Main memory	32 KiB first-fit heap, split/free/coalesce, fit comparison
9	Virtual memory	Per-process pages, six frames, FIFO trace, LRU/OPT comparison
10	File interface	Hierarchical MiniFS: list, mkdir, cd, read, write, remove
11	File implementation	Metadata statistics and allocation-method comparison
12	Mass storage	FCFS, SSTF, SCAN, C-SCAN, C-LOOK, and RAID comparison
13	I/O systems	Polling, interrupts, DMA, blocking modes, and counters

4. Interface and Presentation Design

The graphical desktop uses a left sidebar and movable windows. Files and Notes share MiniFS; Task Manager exposes live scheduler state; Calculator and Clock demonstrate standalone apps; Settings changes wallpaper and scheduling policy; OS Lab provides chapter buttons. The terminal uses high-

contrast output, adaptive line wrapping, scroll controls, color-coded status messages, and a large command entry area.

For assessment, the command 'teacher' opens a recommended tour. 'explain CH' provides talking points without changing state, while 'present CH' states the concept, identifies the exact kernel implementation, and runs live proof. The red STOP button and 'stop' command terminate all non-shell background tasks, including named labs and standalone counters, while preserving a responsive shell.

5. Challenges Using the STAR Format

Challenge 1: Porting from ESP32-S3 to a bootable x86 system

Situation. The earlier mini-kernel depended on an ESP32-S3 runtime and serial-oriented workflow, while the submission needed to boot as a PC ISO in Oracle VirtualBox.

Task. Recreate the learning features without an operating-system host, C standard library, or ESP32 services, and provide visible keyboard, mouse, and graphical output.

Action. I wrote a BIOS/El Torito loader, entered protected mode through GDT and A20 setup, passed VESA framebuffer information to a freestanding C kernel, and implemented IDT/PIC/PIT, serial, PS/2 input, graphics, and a custom runtime library.

Result. JAS OS boots directly from a 1.44 MB ISO image and provides an interactive 1024x768 desktop in VirtualBox. The final kernel and ISO build successfully with the cross-compiler and no warnings.

Challenge 2: Turning slide concepts into observable kernel behavior

Situation. A list of textbook algorithms would satisfy names but would be difficult to defend as an implemented operating system during a live demonstration.

Task. Connect each major chapter to stateful code and show evidence that changes while the kernel runs.

Action. I separated tasks, scheduling, synchronization, memory, paging, deadlock avoidance, files, storage, and I/O into kernel modules. I added process tables, timelines, counters, safe sequences, page-fault traces, allocation statistics, and event logs as observable outputs.

Result. Chapters 1-13 now have a direct command and OS Lab entry. The live output exposes task IDs, states, run slices, semaphore values, frame ownership, head movement, and I/O counts instead of only displaying definitions.

Challenge 3: Making a complex terminal easy to explain

Situation. The growing command set was complete but too dense to remember under assessment conditions, and asynchronous output could hide the most relevant evidence.

Task. Create a teacher-friendly path that explains what a feature proves before it runs and remains readable at the VM resolution.

Action. I redesigned the terminal with a high-contrast viewport, toolbar, scroll position, large prompt, status highlighting, and Guide/Lectures controls. I added 'teacher', 'explain CH', and 'present CH', then connected all OS Lab buttons to the same presentation flow.

Result. A student can now follow one short guide or click a chapter, read the concept and implementation, and immediately show live proof. The 4:17 narrated 1080p submission video uses this exact flow.

Challenge 4: Stopping concurrent terminal activity safely

Situation. Producer-consumer, thread, readers-writers, and counter tasks can write asynchronous output. The old 'stop' command controlled only one demonstration, which made recovery confusing.

Task. Provide one dependable emergency stop without terminating the terminal or leaving internal lab flags inconsistent.

Action. I changed 'stop' into a global handler that first calls each lab's normal shutdown routine, then scans the task table and terminates every remaining non-shell task. It restores the scheduler's global state and reports what was stopped. Specific commands such as 'demo stop', 'thread stop', and 'rw stop' remain available.

Result. The red STOP button, 'stop', 'stop all', and 'lab stop' now end all background terminal activity while the shell stays READY and continues accepting commands.

6. Verification and Results

Check	Verified result
Compiler	i686-elf-gcc with -ffreestanding, -Wall, -Wextra, -O2
Build result	Clean build; no warnings or errors
Kernel image	92,384 bytes (JAS OS v1.5.3)
Bootable ISO	1,523,712 bytes; SHA-256 2B5A69F0D4CF0BC80321FCA6C608B1B73E16B44074966462E49EEB870043B6EF
VM configuration	Other/Unknown 32-bit, BIOS, 512 MB RAM, PS/2 mouse, DVD first
Runtime evidence	Real 1024x768 screenshots and a 2:26 VirtualBox WebM demonstration

7. Demonstration Procedure

Opening - teacher

Show the assessment route and project scope.

Kernel - present 1

Show kernel identity and live resource state.

Processes - present 3

Demonstrate IPC and the process table.

Scheduling - present 5

Show the active policy and recent CPU timeline.

Synchronization - present 6

Start readers-writers and expose protected shared data.

Emergency control - stop

Stop every background task while retaining the shell.

Safety - present 7

Show the Banker's safe sequence.

Memory - present 9

Show demand paging and FIFO frame activity.

Storage and I/O - present 12; present 13

Compare disk policies and device workflows.

8. Limitations and Future Work

JAS OS is an educational kernel, not a production operating system. Tasks are cooperative step functions rather than preemptive user processes; applications execute in kernel space; MiniFS is RAM-backed for the VM session; and paging, disk, RAID, and I/O commands are transparent simulations. These choices keep the code small enough to explain while preserving the algorithms and state changes required by the course. Future work could add ring-3 processes, a hardware-backed disk file system, preemptive context switching, an ELF program loader, and network drivers. Protection and security would begin at chapter 14 and are intentionally not claimed in this submission.

Conclusion

JAS OS meets the main project motive: it converts operating-system knowledge from chapters 1-13 into a system that boots, runs, changes state, accepts commands, and can be explained clearly. The project demonstrates low-level x86 boot engineering, freestanding C, interrupts and devices, kernel data structures, algorithm implementation, GUI design, presentation design, and disciplined scope.